TARGETED 12-WEEK ROADMAP

Architecture Preparation Roadmap

Azure · Distributed Systems · Agentic AI · FDE Readiness

Project-led curriculum
Current certification strategy
Curated primary-source library
Production-readiness scorecards

Iryna Tyshchenko · October 2026

Contents

Use the roadmap sequentially. Use the resource library selectively. Certifications validate demonstrated ability; they do not replace architecture evidence.

How to use this guide

1. Target capability model
   - 1.1 What an architect must demonstrate
   - 1.2 Evidence portfolio

2. Twelve-week project-led roadmap
   - Week 1 — Requirements, constraints, and architecture framing
   - Week 2 — Architecture decisions and service-selection discipline
   - Week 3 — Data architecture and consistency
   - Week 4 — APIs, events, and third-party integrations
   - Week 5 — Distributed workflows and messaging
   - Week 6 — Azure platform, networking, identity, and governance
   - Week 7 — Reliability, observability, operations, and cost
   - Week 8 — Performance and evidence-driven architecture
   - Week 9 — RAG and governed AI applications
   - Week 10 — Agentic systems and safe actions
   - Week 11 — Architecture review and stakeholder communication
   - Week 12 — Capstone defence and interview readiness

3. Certification strategy for current architect roles
   - 3.1 Market reading
   - 3.2 Priority 1 — Azure Solutions Architect Expert
   - 3.3 Priority 2 — Azure AI Apps and Agents Developer Associate
   - 3.4 Priority 3 — Agentic AI Business Solutions Architect Expert
   - 3.5 Specialization credentials
   - 3.6 Recommended certification order
   - 3.7 What not to prioritize

4. Targeted resource library
   - P0 — Use throughout the roadmap
   - P1 — Use for specific gaps
   - P2 — Optional interview support

5. Weekly operating rhythm
   - Monday — Learn and frame
   - Tuesday — Design
   - Wednesday — Build
   - Thursday — Review
   - Friday — Explain
   - Weekend — Certification lane

6. Architecture review scorecard

7. Final production-readiness checklist
   - Design
   - Security
   - Reliability
   - Delivery and operations
   - AI quality

8. The shortest effective path
   - Sources

How to use this guide

This is not a list to memorize. It is a decision-making curriculum. Every week produces evidence that you can clarify an ambiguous problem, select a safe default, identify failure boundaries, explain trade-offs, and revise the design after feedback.

Use one coherent capstone throughout: a **C#/.NET commerce platform with an AI customer-operations agent**. The platform accepts orders, reserves inventory, processes payments, coordinates delivery, answers support questions from governed enterprise data, and escalates consequential actions to a human.

For every architectural decision, answer five questions:

1. What requirement drives this choice?
2. What is the safe default?
3. Under which measurable conditions does it stop fitting?
4. What alternative would replace it?
5. What test or operational signal validates the decision?

The Azure Architecture Center is the main reference library; the Well-Architected Framework provides the quality model, and Cloud Design Patterns supplies reusable implementation patterns.[1][2][3]

---

1. Target capability model

1.1 What an architect must demonstrate

Problem framing

- Translate business goals into functional and non-functional requirements.
- Expose constraints, assumptions, dependencies, and unresolved questions.
- Define measurable targets for latency, throughput, availability, recovery, security, privacy, and cost.
- Distinguish a reversible experiment from a high-cost irreversible decision.

System design

- Set system boundaries and ownership.
- Select compute, storage, integration, networking, and identity approaches.
- Design for failure rather than only for the happy path.
- Describe consistency, ordering, idempotency, retries, and reconciliation precisely.
- Plan deployment, observability, rollback, and incident ownership.

AI architecture

- Separate deterministic workflows from model-driven judgment.
- Ground model responses in governed data.
- Constrain tool permissions and propagate user identity.
- Evaluate quality, safety, latency, and cost continuously.
- Require human approval for high-impact or irreversible actions.

FDE and consulting behavior

- Discover the real customer problem before proposing technology.
- Build a thin vertical slice quickly.
- Explain the same design differently to executives, product leaders, security teams, and engineers.
- Leave a maintainable system, decision record, and operating model—not only a demo.

1.2 Evidence portfolio

By the end, your repository should contain:

- 1 system context diagram
- 3 container/deployment diagrams
- 6 sequence or state diagrams
- 8–12 Architecture Decision Records (ADRs)
- API and event contracts
- Threat model
- Reliability targets and failure-mode analysis
- Cost model
- Load-test report
- AI evaluation dataset and results
- Operations runbook
- 10-minute executive presentation
- 30–60 second interview answer for every major decision

Use the C4 model for diagrams and ADRs for decisions. C4 encourages multiple abstraction levels; ADRs preserve context, options, consequences, and status.[5][6]

---

2. Twelve-week project-led roadmap

Week 1 — Requirements, constraints, and architecture framing

Learn

- Functional versus non-functional requirements
- Quality attributes and architectural drivers
- Stakeholder mapping
- Assumptions, risks, and decision reversibility
- Capacity-estimation fundamentals
- C4 system context diagrams

Build

Write a two-page brief for the commerce platform:

- 100,000 monthly active users
- bursty checkout traffic
- external payment and delivery providers
- regulated customer data
- AI support agent with read and write tools
- explicit cost ceiling

Create:

- System context diagram
- Stakeholder map
- Assumption log
- Initial risk register
- Capacity estimate

Completion test

You can explain why the architecture cannot be selected before availability, consistency, data residency, latency, integration, and cost constraints are known.

Primary resources

- Azure Architecture Center[1]
- Azure Well-Architected Framework[2]
- C4 diagrams[5]

Week 2 — Architecture decisions and service-selection discipline

Learn

- Safe defaults versus premature complexity
- Build versus buy
- Managed service versus self-managed platform
- Coupling, cohesion, modular monoliths, and microservices
- ADR structure and lifecycle
- Technology-selection benchmarks

Build

Write ADRs for:

1. Modular monolith versus microservices
2. Azure Container Apps versus AKS
3. Azure SQL versus Cosmos DB
4. Synchronous versus asynchronous order workflow

For each ADR include context, options, decision, positive and negative consequences, exit criteria, and verification plan.

Completion test

You can defend Container Apps as a default while naming the operational, networking, scheduling, or platform-control requirements that could justify AKS. Microsoft's compute and container-selection guides should inform the comparison rather than product familiarity alone.[32][33]

Primary resources

- Maintain an ADR[6]
- Compute decision tree[32]
- Container-service considerations[33]
- *Fundamentals of Software Architecture*
- *Software Architecture: The Hard Parts*

Week 3 — Data architecture and consistency

Learn

- Relational modelling, normalization, and indexes

- Transactions and isolation
- Optimistic concurrency
- Replication and partitioning
- Eventual consistency
- Caching and invalidation
- Change data capture and audit history
- Vector retrieval as a specialized index, not a source of truth

Build

Design:

- Order, inventory, payment, and shipment data models
- Transaction boundaries
- Cache policy
- Audit/event history
- Data-retention policy
- RAG ingestion path from approved support content

Create failure scenarios for duplicate checkout requests, stale inventory, payment timeout with unknown outcome, and delayed events.

Completion test

You can state exactly where strong consistency is required, where eventual consistency is acceptable, and how reconciliation restores correctness.

Primary resources

- *Designing Data-Intensive Applications, 2nd Edition*[7]
- Amazon Builders' Library[10]
- MIT distributed-systems material for deeper study[8]

Week 4 — APIs, events, and third-party integrations

Learn

- REST resource and operation modelling
- OpenAPI and AsyncAPI contracts
- API versioning and pagination
- Idempotency keys
- Webhook signatures and replay prevention

- Timeouts, retries, backoff, jitter, and circuit breakers
- Outbox/inbox patterns
- Durable deduplication
- Long-running operations
- Reconciliation after unknown outcomes

Build

Create:

- Checkout REST contract
- Payment webhook contract
- Order event schema
- Delivery-provider adapter
- Idempotency and deduplication design
- Sequence diagram showing payment timeout, retry, duplicate callback, and reconciliation

Microsoft publishes REST API guidelines, while AsyncAPI standardizes event-driven interface descriptions.[11] [12]

Completion test

You never claim end-to-end exactly-once effects without defining the transaction boundary. You can explain that a timeout means “outcome unknown,” not “operation failed.”

Primary resources

- Microsoft REST API Guidelines[11]
- AsyncAPI[12]
- OWASP API Security[39]
- Azure Cloud Design Patterns[3]

Week 5 — Distributed workflows and messaging

Learn

- Queues versus streams
- Competing consumers
- Partitioning and per-aggregate ordering
- Backpressure and load levelling
- Dead-letter queues
- Saga choreography versus orchestration

- Compensation
- Poison-message handling
- Consumer idempotency

Build

Implement a minimal order workflow using C#/.NET and Azure Service Bus or an equivalent local broker:

1. Order accepted
2. Inventory reserved
3. Payment authorized
4. Shipment requested
5. Compensation if payment or inventory fails

Record `orderId`, `operationId`, `eventId`, and `version` across every step. Add bounded retries and a reconciliation worker.

Completion test

You can explain ordering scope, duplicate handling, poison-message isolation, and recovery after a consumer crashes between side effect and acknowledgement.

Primary resources

- Cloud Design Patterns[3]
- *Enterprise Integration Patterns*
- *Designing Data-Intensive Applications*[7]
- MIT 6.5840 Distributed Systems[8]

Week 6 — Azure platform, networking, identity, and governance

Learn

- Subscriptions, management groups, policy, tags, and budgets
- Landing zones and environment separation
- Entra ID, managed identities, workload identities, and RBAC
- Key Vault and secretless design
- Private networking, Private Link, DNS, and egress control
- Container Apps, Functions, App Service, and AKS selection
- Infrastructure as Code with Bicep or Terraform

Build

Produce an Azure deployment design with:

- Separate development, test, and production environments
- Managed identities instead of embedded credentials
- Private data-plane connectivity
- Least-privilege roles
- Policy controls and cost budgets
- Infrastructure-as-Code skeleton

The Cloud Adoption Framework adds organization, governance, and landing-zone context that an application-only design misses.[4]

Completion test

You can trace one user request through identity, network, compute, data, secrets, monitoring, and policy boundaries.

Primary resources

- Cloud Adoption Framework[4]
- Azure Architecture Center[1]
- Well-Architected Framework[2]
- Azure pricing calculator[37]

Week 7 — Reliability, observability, operations, and cost

Learn

- SLI, SLO, and error budget
- Availability and dependency budgets
- RTO and RPO
- Multi-zone and multi-region decisions
- Health models and graceful degradation
- Metrics, logs, traces, correlation IDs, and business telemetry
- Deployment strategies and rollback
- FinOps and unit economics

Build

Create:

- Service-level objectives

- Dependency map
- Failure-mode and effects analysis
- Dashboard specification
- Alert-routing rules
- Rollback procedure
- Disaster-recovery exercise
- Cost per 1,000 orders and per AI support conversation

Microsoft provides explicit reliability-review, threat-modelling, and observability guidance; use the checklists as design review inputs, not as compliance theatre.[34][35][36]

Completion test

You can explain what the customer experiences when each dependency fails and which signal triggers which owner action.

Primary resources

- Google SRE books[9]
- Reliability checklist[34]
- Observability strategy[36]
- *Release It!*

Week 8 — Performance and evidence-driven architecture

Learn

- Workload modelling
- Baselines and bottleneck identification
- Load, stress, soak, and failover tests
- Queuing effects
- Autoscaling signals
- Database and cache profiling
- Cost/performance trade-offs

Build

Run a load test against checkout and order-status paths. Report:

- p50, p95, and p99 latency
- throughput and error rate
- resource saturation

- queue depth
- scaling behaviour
- cost change
- failure threshold

Azure Load Testing can support repeatable load and resilience validation.[38]

Completion test

Every major scale claim in your design is backed by a measurement, calculation, or explicit assumption.

Primary resources

- Azure Load Testing[38]
- Google SRE books[9]
- Amazon Builders' Library[10]

Week 9 — RAG and governed AI applications

Learn

- Source-of-truth boundaries
- Ingestion, chunking, metadata, embeddings, and retrieval
- Access-control-aware retrieval
- Hybrid search and reranking
- Citations and grounding
- Evaluation datasets
- Quality, latency, and cost trade-offs
- Prompt injection and data exfiltration

Build

Create an AI support assistant that:

- Answers from approved product and order-policy content
- Retrieves only content the requesting identity may access
- Provides citations
- Refuses unsupported claims
- Records retrieval and model telemetry
- Runs a repeatable evaluation set

Microsoft's RAG guidance treats design and evaluation as an end-to-end architecture problem rather than only a prompt problem.[13]

Completion test

You can separate retrieval failure, generation failure, authorization failure, and source-data failure in metrics and incident handling.

Primary resources

- Agentic RAG on Azure[13]
- Semantic Kernel[14]
- OWASP LLM and GenAI Top 10[16]

Week 10 — Agentic systems and safe actions

Learn

- Workflow versus autonomous agent
- Tool contracts and permission scoping
- Planning and orchestration
- Memory boundaries
- Human approval
- Durable execution
- Compensating action
- Agent evaluation and trace review
- Model routing and fallback

Build

Extend the support assistant so it can propose—but not autonomously execute—refunds. Add:

- Tool allowlist
- Per-user authorization
- Maximum refund limit
- Human approval
- Idempotency key
- Audit trail
- Timeout and unknown-outcome handling
- Evaluation cases for prompt injection, unauthorized refund, duplicate execution, and tool failure

OpenAI's agent guide describes agents as systems combining models, tools, and instructions; the architecture must constrain all three.[15]

Completion test

You can identify which decisions are deterministic, which are model-driven, which are reversible, and which require approval.

Primary resources

- Practical guide to building agents[15]
- Azure agentic RAG guidance[13]
- Semantic Kernel documentation[14]
- OWASP LLM guidance[16]

Week 11 — Architecture review and stakeholder communication

Learn

- Executive-first communication
- Architecture review facilitation
- Decision framing
- Handling disagreement
- Risk acceptance
- Roadmaps and migration sequencing
- Presenting cost and business impact

Build

Prepare three versions of the same design:

- 90-second executive explanation
- 10-minute architecture-review presentation
- Detailed engineering handover

Run an architecture kata with a mentor or peer. Architectural Katas are designed specifically for practising architecture through realistic scenarios.[40]

Completion test

You can lead with the decision, impact, evidence, risk, and ask—without walking executives through every component.

Primary resources

- Architectural Katas[40]
- *The Staff Engineer's Path*
- *Team Topologies*
- ADR guidance[6]

Week 12 — Capstone defence and interview readiness

Final scenario

A retailer wants an AI-assisted customer-operations platform in eight weeks. It must integrate with legacy order systems, process payment and refund operations safely, support multiple countries, protect personal data, survive partner outages, and remain within a fixed Azure budget.

Deliverables

- Requirements and assumptions
- Context, container, sequence, state, and deployment diagrams
- Data model and consistency strategy
- API and event contracts
- ADR set
- Threat model
- Reliability and disaster-recovery plan
- Cost estimate
- Load-test report
- AI evaluation report
- Operations runbook
- Migration roadmap
- Executive presentation

Defence questions

- Why not use microservices immediately?
- Why this database?
- What does a payment timeout mean?
- Where can duplicate business effects occur?
- What fails if the model or vector index is unavailable?
- How is identity propagated into agent tools?
- What action requires human approval?

- What is the first scaling limit?
- What would you remove if the budget were cut by 40%?
- What evidence would make you reverse the decision?

---

3. Certification strategy for current architect roles

3.1 Market reading

A targeted scan of current Azure and AI architecture postings—not a statistical census—shows **Azure Solutions Architect Expert (AZ-305)** recurring consistently in sampled roles.[26][27]

Current postings also mention AZ-104, AZ-400, AI-103, AB-100, security credentials, Terraform, and Kubernetes according to specialization.[28][29]

Do not optimize for collecting badges. Use certifications to validate a portfolio that already shows design and delivery ability.

3.2 Priority 1 — Azure Solutions Architect Expert

Path

1. **Azure Administrator Associate (AZ-104)**
2. **AZ-305: Designing Microsoft Azure Infrastructure Solutions**
3. Earn **Microsoft Certified: Azure Solutions Architect Expert**

Microsoft currently lists Azure Administrator Associate as the prerequisite certification and AZ-305 as the required architect exam.[17][31]

Why prioritize it

- Directly aligned with Azure Solutions Architect and Cloud Architect titles
- Covers identity, governance, monitoring, data, continuity, and infrastructure design
- Commonly named in Azure architecture postings from consultancies and enterprise employers[26][27][28]

Evidence required beyond the exam

- Landing-zone design
- Identity and network boundaries
- Migration plan

- Availability and disaster recovery
- Cost model
- Well-Architected review

3.3 Priority 2 — Azure AI Apps and Agents Developer Associate

Current exam

AI-103: Developing AI Apps and Agents on Azure

Microsoft positions AI-103 around planning and managing Azure AI solutions and implementing generative-AI and agentic systems with Microsoft Foundry.[18]

Important update

Do **not** start new preparation for AI-102. Microsoft retired AI-102 on 30 June 2026; AI-103 is the current replacement path.[30]

Why prioritize it

- Strong fit for AI architect and FDE roles
- Forces hands-on implementation rather than architecture vocabulary alone
- Supports a credible agentic-AI portfolio

Evidence required beyond the exam

- Working agent
- RAG pipeline
- Evaluation dataset
- Security and authorization tests
- Observability and cost report

3.4 Priority 3 — Agentic AI Business Solutions Architect Expert

Current exam

AB-100: Agentic AI Business Solutions Architect

The expert credential requires AB-100 plus an eligible associate-level prerequisite such as AI-103. Microsoft describes the role around AI-driven business solutions, cross-platform integration, responsible AI, and multi-agent orchestration.[19]

When to pursue

Pursue after AI-103 if targeting:

- AI Solutions Architect
- Copilot/Power Platform architecture
- Agentic transformation consulting
- Customer-facing Microsoft solution roles

Do not place AB-100 ahead of core cloud architecture and hands-on agent delivery.

3.5 Specialization credentials

DevOps and platform architecture

AZ-400: DevOps Engineer Expert — valuable when roles emphasize CI/CD, GitHub Actions, Azure DevOps, infrastructure delivery, compliance, and observability. Microsoft requires AZ-400 plus an eligible associate credential.[20]

Cloud and AI security

SC-500: Cloud and AI Security Engineer Associate — the current implementation-oriented security credential for Azure, hybrid, and AI workloads.[21]

SC-100: Cybersecurity Architect Expert — useful for security architecture, Zero Trust, regulated industries, and governance-heavy roles. It requires SC-100 plus an eligible associate security credential such as SC-500.[22]

Do not start AZ-500: it retired in August 2026 and has been replaced by the newer security path.[21]

Infrastructure as Code

HashiCorp Terraform Associate — useful when target roles explicitly request Terraform or multi-cloud IaC. The current Associate exam validates foundational Terraform workflows, state, configuration, and IaC concepts.[24]

Kubernetes

CKA — pursue only when target roles require AKS/Kubernetes platform depth. The CNCF exam is a two-hour performance-based command-line assessment.[23]

Enterprise architecture

TOGAF Enterprise Architecture Foundation/Practitioner — useful for enterprise architecture, consulting, governance, capability mapping, and formal architecture-board roles. The Open Group's current portfolio includes TOGAF Standard 10th Edition paths.[25]

3.6 Recommended certification order

Core Azure/FDE route

1. AZ-104, unless already held or genuinely mastered

2. AZ-305
3. AI-103
4. AB-100

Platform-heavy route

1. AZ-104
2. AZ-305
3. AZ-400
4. Terraform Associate
5. CKA only if job descriptions require Kubernetes

Security-heavy route

1. AZ-104
2. AZ-305
3. SC-500
4. SC-100

Enterprise-consulting route

1. AZ-104
2. AZ-305
3. TOGAF Enterprise Architecture Foundation/Practitioner
4. AI-103 or AB-100 depending on delivery focus

3.7 What not to prioritize

- Fundamentals badges when you already possess demonstrable professional experience, unless a specific employer requires them
- Retired AI-102 or AZ-500 preparation[21][30]
- Multi-cloud certifications without a target role that values them
- CKA without hands-on Kubernetes responsibility
- TOGAF as a substitute for cloud, data, integration, and operational design

4. Targeted resource library

P0 — Use throughout the roadmap

Architecture and Azure

- Azure Architecture Center[1]
- Azure Well-Architected Framework[2]
- Azure Cloud Design Patterns[3]
- Cloud Adoption Framework[4]
- C4 Model[5]
- ADR guidance[6]

Distributed systems and reliability

- *Designing Data-Intensive Applications, 2nd Edition*[7]
- MIT 6.5840 Distributed Systems[8]
- Google SRE books[9]
- Amazon Builders' Library[10]

APIs and integration

- Microsoft REST API Guidelines[11]
- AsyncAPI[12]
- OWASP API Security[39]
- *Enterprise Integration Patterns*
- *Release It!*

AI and agents

- Azure agentic-RAG architecture guide[13]
- Semantic Kernel documentation[14]
- OpenAI practical agent guide[15]
- OWASP LLM and GenAI Top 10[16]

P1 — Use for specific gaps

Software architecture books

- *Fundamentals of Software Architecture*
- *Software Architecture: The Hard Parts*

- *Building Evolutionary Architectures*
- *Building Microservices*
- *Monolith to Microservices*
- *Domain-Driven Design Distilled*

Leadership and organization

- *The Staff Engineer's Path*
- *Team Topologies*
- *Accelerate*

Practice

- Architectural Katas[40]
- System Design Primer
- Azure reference architectures
- Weekly mentor review
- Mock architecture-review panels

P2 — Optional interview support

- ByteByteGo for visual refreshers
- Hello Interview for structured mock practice
- Educative or Design Gurus for guided question sets
- Exponent for interview demonstrations

Treat commercial solutions as examples to critique. Never memorize a diagram without reconstructing it from requirements.

---

5. Weekly operating rhythm

Monday — Learn and frame

- 60–90 minutes of primary-source reading
- Rewrite the problem in your own words
- Identify three unknowns

Tuesday — Design

- Produce one diagram
- Write one ADR
- Add failure paths

Wednesday — Build

- Implement the thinnest vertical slice
- Record telemetry
- Trigger one controlled failure

Thursday — Review

- Present to a mentor or peer
- Capture objections without defending immediately
- Revise the design

Friday — Explain

Produce:

- 30-second answer
- 5-minute engineering explanation
- One executive summary
- One “what would change my decision?” statement

Weekend — Certification lane

Use certification study guides to detect gaps, then close each gap through a lab. Avoid exam-dump preparation.

---

6. Architecture review scorecard

Score each dimension from 0 to 2:

- Requirements are measurable

- Constraints and assumptions are explicit
- Boundaries and ownership are clear
- Data consistency is justified
- API and event contracts are versioned
- Retries are bounded and idempotent
- Unknown outcomes have reconciliation
- Security is layered
- AI tools inherit least privilege
- Reliability targets map to mechanisms
- Observability covers technical and business outcomes
- Cost scales with workload assumptions
- Deployment and rollback are defined
- Alternatives and exit criteria are documented
- The design can be explained to multiple audiences

Interpretation:

- 24–30: ready for a formal review
- 17–23: credible but has important gaps
- 10–16: architecture vocabulary without enough evidence
- Below 10: return to requirements and boundaries

---

7. Final production-readiness checklist

Design

- ☐ Requirements and constraints are approved
- ☐ Context and deployment diagrams are current
- ☐ ADRs record major decisions and exit criteria
- ☐ Data ownership and consistency are explicit
- ☐ External dependencies have isolation and reconciliation

Security

- ☐ Threat model completed
- ☐ Managed identity and least privilege used

- ☐ Secrets are not embedded
- ☐ Data classification and retention defined
- ☐ Agent tool permissions are scoped
- ☐ Prompt-injection and exfiltration tests exist

Reliability

- ☐ SLOs and dependency budgets defined
- ☐ Retry, timeout, backoff, and idempotency policies documented
- ☐ Dead-letter and poison-message procedures tested
- ☐ RTO/RPO and recovery exercise completed
- ☐ Graceful degradation defined

Delivery and operations

- ☐ Infrastructure is reproducible
- ☐ CI/CD includes security and quality gates
- ☐ Rollback tested
- ☐ Metrics, logs, traces, and business telemetry available
- ☐ Alerts have owners and runbooks
- ☐ Cost budgets and anomaly alerts configured

AI quality

- ☐ Evaluation dataset versioned
- ☐ Grounding and citation quality measured
- ☐ Authorization evaluated separately from answer quality
- ☐ Model and retrieval failures distinguishable
- ☐ Human approval protects consequential actions
- ☐ Audit records connect user, model, tool, and outcome

---

8. The shortest effective path

If time is limited, do these ten things:

1. Study the Azure Well-Architected Framework.[2]

2. Learn C4 and ADRs.[5][6]
3. Read the most relevant chapters of *Designing Data-Intensive Applications*.[7]
4. Design one payment workflow with unknown-outcome recovery.
5. Build one C#/.NET asynchronous workflow.
6. Deploy it with identity, networking, monitoring, and cost controls.
7. Add one governed RAG path.
8. Add one agent tool with human approval and idempotency.
9. Run a load test and architecture review.
10. Complete AZ-104/AZ-305 and AI-103 in the order appropriate to your existing credentials.[17][18][31]

The goal is not to know every possible architecture. The goal is to make a defensible decision, identify where certainty ends, validate the risky assumptions, and communicate the result clearly.

Sources

- **[1]** learn.microsoft.com — azure / architecture
- **[2]** learn.microsoft.com — well-architected / what-is-well-architected-framework
- **[3]** learn.microsoft.com — architecture / patterns
- **[4]** learn.microsoft.com — cloud-adoption-framework / overview
- **[5]** c4model.com — diagrams.html
- **[6]** learn.microsoft.com — architect-role / architecture-decision-record
- **[7]** www.oreilly.com — designing-data-intensive / 9781098119058
- **[8]** pdos.csail.mit.edu — 6.824 / index.html
- **[9]** sre.google — books
- **[10]** builder.aws.com — topics / builders-library
- **[11]** github.com — microsoft / api-guidelines
- **[12]** www.asyncapi.com — home
- **[13]** learn.microsoft.com — rag / rag-agentic
- **[14]** learn.microsoft.com — en-us / semantic-kernel
- **[15]** cdn.openai.com — business-guides-and-resources / a-practical-guide-to-building-agents.pdf
- **[16]** genai.owasp.org — initiatives / top-10-for-llm-and-genai
- **[17]** learn.microsoft.com — certifications / azure-solutions-architect
- **[18]** learn.microsoft.com — certifications / azure-ai-apps-and-agents-developer-associate
- **[19]** learn.microsoft.com — certifications / agentic-ai-business-solutions-architect
- **[20]** learn.microsoft.com — certifications / devops-engineer
- **[21]** learn.microsoft.com — certifications / cloud-and-ai-security-engineer-associate
- **[22]** learn.microsoft.com — certifications / cybersecurity-architect-expert
- **[23]** www.cncf.io — certification / cka
- **[24]** developer.hashicorp.com — certifications / terraform-associate
- **[25]** www.opengroup.org — certifications / togaf-certification-portfolio
- **[26]** jobs.lever.co — lvs1 / 6e441599-b9a0-43bb-8f05-a383004e1f4e

- **[27]** jobs.lever.co — fp / a6addff8-f69a-4b0f-86db-d3877ed6c2d5
- **[28]** jobs.lever.co — atmosera / 20ee35c1-e7d0-4cc8-99d3-8d3c5c1b0677
- **[29]** jobs.lever.co — thinkahead / 3595a68c-61a7-4d0c-91d4-914fa9803cbd
- **[30]** learn.microsoft.com — study-guides / ai-102
- **[31]** learn.microsoft.com — certifications / azure-administrator
- **[32]** learn.microsoft.com — technology-choices / compute-decision-tree
- **[33]** learn.microsoft.com — guide / container-service-general-considerations
- **[34]** learn.microsoft.com — reliability / checklist
- **[35]** learn.microsoft.com — security / threat-model
- **[36]** learn.microsoft.com — operational-excellence / observability
- **[37]** azure.microsoft.com — pricing / calculator
- **[38]** learn.microsoft.com — azure / load-testing
- **[39]** owasp.org — API-Security
- **[40]** www.architecturalkatas.com — about.html